

Python Code Review

Goal

Coordinate several LLMs, give each one a job description and simulate the same process a human follows before deciding to buy, hold or sell a stock.

Structure of the program

Continuous flow of information:

1. Four analysts collect data.
 - **Market analyst** (*market_analyst.py*) checks price data and technical indicators. Functions used - *get_stock_data* and *get_indicators*. Produces *market_report*.
 - **News analyst** (*news_analyst.py*) reads company and macro news. Functions used – *get_news* and *get_global_news*. Produces *news_report*.
 - **Fundamental analyst** (*fundamentals_analyst.py*) goes through financial statements. Functions used - *get_fundamentals*, *get_balance_sheet*, *get_cashflow* and *get_income_statement*. Produces *fundamentals_report*.
 - **Sentiment analyst** (*sentiment_analyst.py*) measures social-media sentiment – fetches news, stocktwits, reddit. Produces *sentiment_report*.
2. Then **bull analyst** (*bull_researcher.py*) and **bear analyst** (*bear_researcher.py*) argue over what that material means. Produce *investement_debate_state*.
3. **Research manager** (*research_manager.py*) reads the argument and writes an investment plan. Produces *investement_plan*.
4. **Trader** (*trader.py*) turns the plan into a decision to buy, hold or sell. Produces *trader_investement_plan*.
5. **Aggressive, conservative and neutral analysts** (*aggressive_debator.py*, *conservative_debator.py*, *neutral_debator.py*) debate over the decision from a risk perspective. Produces *risk_debate_state*.
6. **Portfolio manager** (*portfolio_manager.py*) weighs the risk debate and gives the final rating. Produces *final_trade_decision*.
7. Another memory component (*memory.py*) records every decision.

Each of the above components are in its own file and they all follow the same structure.

Language and libraries

- Everything is in Python
 - Uses standard libraries + four external libraries
1. **LangChain (langchain_core)** talks to the language models.
 - *ChatPromptTemplate*, *MessagesPlaceholder* - provides the prompt templates that are used and filled at a call time. This allows the same structure for different analysts.
 - *@tool* from *langchain_core.tools* - wraps a function so that LLM sees it as something available for it to call.

- Messages like *HumanMessage*, *AIMessage*, *RemoveMessage* from *langchain_core.messages* – classes representing turns and ownership in a conversation with an LLM.
- *llm.invoke()*, *llm.bind_tools()*, *llm.with_structured_output()* - methods added to whatever chat model object is used, not standalone functions

Overall: handles prompting, tool exposure and conversation typing. Basically it's the layer that standardizes how this codebase talks to LLM.

2. **LangGraph** runs the agents in order. It models all components as a graph of nodes, where each node is one of those roles, defining which role runs after which.
 - *AgentState(MessagesState)* (*agent_states.py*) – dictionary-like shared object. Nodes do not call each other, instead they read from and write to a single state.
 - The state is carried from one node to the next – each node returns something *like* *{"market_report": report}* or *{"risk_debate_state": new_risk_debate_state}*. This value is taken and merged into the main shared state object, filling it with more info every time.

Overall: provides **independence** of the agent nodes (none of them call each other) and handles the **orchestration** that decides the run order and carries the shared state.

3. **Pydantic (in schemas.py)** is used to describe the exact shape of the answers the agents must produce. Allows the prompt in *portfolio_manager.py* to stay focused on context (the debate history, the trader's plan) rather than formatting rules.
 - *BaseModel* - the base class every schema inherits from (like *PortfolioDecision(BaseModel)*, *ResearchPlan(BaseModel)*).
 - *Field()* - used to attach a description, default value, or constraints to each schema.

Overall: defines the exact shape an LLM's answer must take.

4. **yfinance (in agent_utils.py)** - only library here that accesses live market data directly. It is used to look up a stock's real company name and sector from its ticker.
 - *yf.Ticker(ticker.upper()).info* - used to look up company longName/ shortName, sector, industry, exchange, quoteType.

Overall: a sanity-check on identity, run before any analysis starts, so the rest of the system works on a verified data.

5. Basic Python modules: *re* (regex, in *rating.py* and *memory.py*) for matching patterns in text, *pathlib* (*memory.py*) and the file system for the memory log, *datetime* (*sentiment_analyst.py*) for working out date ranges, *functools* for caching and for pre-filling a function argument, *logging* and *warnings* for diagnostics.

Separate agent structure

Every role file has the same pattern:

- An outer function (e.g. *create_market_analyst(llm)*), that takes a language model and returns an inner function, usually named something like *market_analyst_node(state)*. The outer function's only job is to build and hand back a ready-to-use node.
- An inner function that is the thing LangGraph runs. The inner function "remembers" the model it was created with, so the model does not have to be called on every call. It is done to configure something once and reuse it.

The single shared state passed between nodes is just a dictionary, which expected fields documented via a *TypedDict* (*AgentState* in *agent_states.py*) with one slot per piece of data like *market_report*, *news_report*, the bull/bear debate, the trader's plan, etc.

Each node reads only the fields it needs and returns just the fields it wants to change; *LangGraph* merges that update into the shared state before passing it to the next node.

Every agent is kept independent, so a new one can be added without rewiring the rest.

Some interesting details about the code that might help later

Data fetching

Three analysts are given tools and allowed to decide what to look up: a tool is a Python function wrapped in *LangChain*'s *@tool* decorator (e.g. *get_stock_data* or *get_income_statement*). The decorator, the function's type hints and its docstring tells the model what the function does and what arguments it expects, so the model can ask to call it correctly.

“Keep working until you are done”

After the model replies, the code checks one condition: *if len(result.tool_calls) == 0* which defines if the report is completed. Only when the model stops asking for tools and simply writes prose is that prose captured as the finished report.

Ways to predict answers from unpredictable model

The project assumes that

- language models are probabilistic: ask the same thing twice and you can get two differently answers
- when information is missing models tend to invent something that sounds believable. A large part of the code exists to fence that behaviour in.

Ways to deal with it:

1. **Structured output schemas** - forces the model's answer into a fixed set of typed fields instead of random form text.
2. **Enums restrict the possible values** - the trader's action can only be Buy/Hold/Sell, and off-format answers are impossible.
3. **Built-in instructions** - the schema tells the model what each field should contain, reducing reliance on wording.
4. **Free-text fallback** (*invoke_structured_or_freetext*) - if output isn't supported or fails, it drops to plain text rather than crashing, so the run still completes.

Data fabrication

Each of the following used to be a bug but got fixed. From the comments in the code:

- **Fabricated sentiment data** - the sentiment analyst was asked to analyze Reddit/ X/ StockTwits but only had a news tool, so the model invented posts that didn't exist Comment:

"Previously named social_media_analyst. Renamed and redesigned because the old version had a prompt that demanded social-media analysis but the only tool available was Yahoo Finance news - which led LLMs to fabricate Reddit/X/StockTwits content under prompt pressure (verified live)."

Was fixed by fetching real data from Yahoo Finance news (via `get_news`), StockTwits messages (`fetch_stocktwits_messages`), and Reddit posts (`fetch_reddit_posts`) instead of letting the model research independently.

- **Invented company identity** – the market analyst would sometimes pattern-match a price chart to the wrong kind of company and invent an identity affecting all later stages. Comment:
- *"This exists to stop the pipeline from hallucinating a different company when a chart pattern suggests a different industry than the real one (#814): without a ground-truth name, the market analyst would pattern-match the price action to a narrative and invent an identity that then cascaded through every downstream agent."*

Was fixed with addition of `yfinance`.

- **"Continue" placeholder hallucination** - the word "Continue" that was supposed to nudge the model after the conversation history got clear got misread and the model would accept it as a user request. Comment:

"The placeholder must not be a bare "Continue": some OpenAI-compatible providers interpret that literally as the user task and produce output about the word 'continue' instead of analysing the instrument (#888)."

Was fixed by replacing with a placeholder that explicitly restates the task.

Memory log pattern

`memory.py` separates log entries using an HTML comment (`<!-- ENTRY_END -->`) instead of something more basic like a blank line.

Apparently, the reason is that an LLM would never write that exact text so it can safely be used. Otherwise, there is a risk of the model's own output accidentally breaking the split causing confusion..